SIMATS ENGINEERING

CO2_AT1_Scenario-Based Assessment

COURSE NAME: Software Testing **COURSE CODE: CSA37**

COURSE OUTCOME COVERED

CO2: *Design effective test cases using black-box, white-box testing techniques and to control methods flow.*

Assessment Tool Weightage: 50%

Code of Conduct:

I P.Ranjithkumar (192411119)___ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: ***P.Ranjithkumar***

1. An online banking system is experiencing frequent failures during fund transfers, especially at boundary values (e.g., minimum and maximum transfer limits). As a test engineer, critically analyze the scenario and evaluate the current test case design strategy. Apply appropriate black box testing techniques such as equivalence partitioning and boundary value analysis to redesign the test cases. Assess the effectiveness of random testing used by the team and justify whether it should be retained or replaced. Propose a structured and optimized testing approach ensuring maximum defect detection. Present your solution with clear reasoning and justification.

Scenario analysis

Frequent transfer failures cluster around minimum and maximum transfer limits. This pattern suggests boundary defects, incorrect comparison operators (< versus ≤), rounding or validation-order problems. A strategy based mainly on ordinary values or random values is unlikely to target these edges consistently.

Relevant theory

Equivalence Class Partitioning (ECP) divides inputs into groups expected to behave similarly; one representative from each class can reduce test count. Boundary Value Analysis (BVA) concentrates on values at and immediately around limits, where off-by-one and inclusive/exclusive errors are common. Random testing can discover unexpected combinations, but without a reproducible seed and boundary weighting it offers weak assurance that important edge cases were covered.

Conceptual flow / architecture

Transfer request → Authenticate → Validate amount and balance → Check daily limit → Execute transfer → Record transaction / return error

Redesigned test cases

Test focus	Input / scenario	Expected result
Amount below minimum	min − smallest unit	Reject; no debit or credit
At minimum	min	Accept if all other rules pass
Just above minimum	min + smallest unit	Accept
Just below maximum	max − smallest unit	Accept
At maximum	max	Accept if maximum is inclusive
Above maximum	max + smallest unit	Reject
Zero / negative	0; −smallest unit	Reject
Insufficient funds	valid amount > available balance	Reject atomically
Daily cap exceeded	amount causes daily total > cap	Reject or require approved workflow

Improved testing strategy

Use ECP for valid amounts, below-minimum, above-maximum, malformed values, insufficient funds and limit-exceeded classes. Apply BVA around both transfer limits using min−δ, min, min+δ, max−δ, max, max+δ, where δ is the smallest permitted currency unit. Add pairwise or risk-based combinations for account status, currency, daily cap and available balance. Retain random testing as a supplementary fuzz/property test, not as the primary design; seed it for reproducibility and bias generation toward boundaries. Include transaction atomicity, duplicate submission and rollback checks.

Coverage, architecture and important checks

Architecture: request/API layer → validation service → limits/balance service → ledger transaction → audit/event log. Assert that rejected transfers create no partial ledger changes. Trace each requirement to tests and use equivalence-class, boundary, negative, integration and regression coverage.

Conclusion / justification

A risk-based, requirement-traceable ECP+BVA suite is more deterministic and efficient than unstructured random testing. Random testing should be retained for discovery after the known classes and boundaries are covered.

1. A healthcare application module has undergone testing, but critical defects were found post-deployment due to insufficient coverage. Evaluate the testing approach used and analyze gaps in test adequacy criteria. Apply white box testing techniques by constructing a conceptual control flow graph and identifying missing paths. Critically assess the coverage achieved (statement, branch, or path coverage). Design an improved testing strategy ensuring complete coverage and justify your decisions. Explain how your approach enhances reliability and reduces risk.

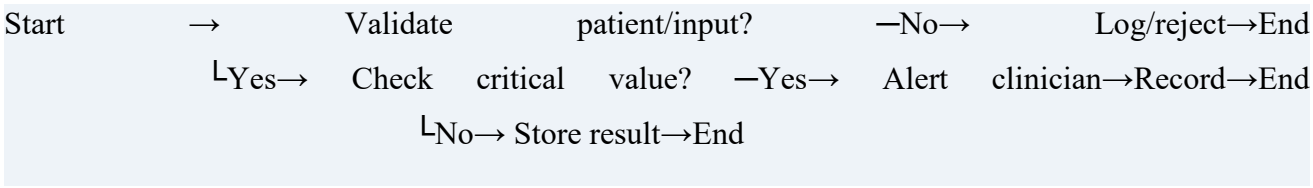
Scenario analysis

Critical defects after deployment indicate that the prior suite did not exercise important decision outcomes or paths. Passing tests alone do not establish adequacy; coverage criteria and risk-based requirements traceability must be examined.

Relevant theory

Statement coverage executes each statement at least once, but can miss an untested decision outcome. Branch coverage executes each decision outcome (true/false), offering stronger decision testing. Path coverage attempts all control-flow paths, but loops and combinations can make complete path coverage infeasible. In healthcare software, safety requirements, abnormal inputs, data integrity and fail-safe behavior deserve explicit tests.

Conceptual flow / architecture



Redesigned test cases

Test focus	Input / scenario	Expected result
Invalid/missing input	Validation=false	Reject safely; no misleading result
Valid, normal result	Validation=true; critical=false	Store correct result
Valid, critical result	Validation=true; critical=true	Alert and record alert
Alert service unavailable	Critical=true; notification fails	Escalate/fallback and log failure
Duplicate/replayed request	Same request twice	Avoid duplicate clinical action; preserve audit trail

Improved testing strategy

Construct a CFG with nodes for validation, critical threshold, persistence and alert handling; edges represent each possible transfer of control. Identify omitted edges such as invalid input, critical result, alert failure, storage failure and retry. Set a minimum of 100% statement and branch coverage for safety-relevant code where feasible, then use targeted path tests for critical workflows, exception handling and recovery. Add data-flow tests for values from input through calculation to stored/displayed result.

Coverage, architecture and important checks

Statement coverage may be high while a false branch or exception path remains untested. Branch coverage exposes decision outcomes; selected path coverage explores meaningful sequences. Full path coverage is often impossible with loops, so document infeasible paths and prioritize hazardous paths using risk analysis.

Conclusion / justification

A CFG-based strategy tied to clinical hazards improves evidence that both normal and failure paths behave safely. Add independent review, integration tests, requirements traceability and post-release monitoring.

3. A retail e-commerce system uses multiple input fields (price, quantity, discount), but testers failed to detect errors in specific input combinations. Analyze the scenario and evaluate the limitations of the current black box testing approach. Apply appropriate techniques such as equivalence class partitioning, boundary value analysis, and other black box methods (e.g., decision table testing). Propose a revised test design strategy covering all possible input combinations. Justify your approach in terms of improved defect detection and test completeness. Provide a structured explanation of your solution.

Scenario analysis

Testing each field independently misses interaction defects—for example, a valid discount combined with a high quantity may exceed a cap or make the total negative. The likely gap is insufficient combination and business-rule testing.

Relevant theory

ECP groups values such as valid/invalid price, valid/invalid quantity and valid/invalid discount. BVA targets price and quantity minima/maxima and discount limits. Decision-table testing is appropriate when output depends on combinations of conditions, because it maps condition combinations to expected actions.

Conceptual flow / architecture

Inputs (price, quantity, discount) → Validate each field → Evaluate discount eligibility/caps → Calculate subtotal and final total → Enforce total ≥ 0 → Display/order

Redesigned test cases

Test focus	Input / scenario	Expected result
Valid baseline	price>0, quantity=1, discount=0	Correct total
Price boundary	0, min positive, max, above max	Reject or calculate per specification
Quantity boundary	0, 1, max, max+1	Reject invalid quantities; cap/accept valid
Discount boundary	0%, max%, >max%	Apply only allowed discount
Cross-field rule	high discount + low subtotal	Never produce negative total

Rounding	fractional prices/discount result	Consistent currency rounding
Promotion eligibility	eligible vs ineligible item/user	Apply only when rule permits

Improved testing strategy

Create ECP classes for each field and apply BVA at every numeric limit. Build a decision table with conditions: item eligible, user eligible, discount within cap, quantity valid, subtotal sufficient; actions: apply discount, reject input, cap discount or calculate total. Add pairwise combinations to control test count, but retain all high-risk rule combinations and invalid interactions. Use an oracle based on an independently reviewed formula, not the same production calculation.

Coverage, architecture and important checks

Test both API and UI validation, plus tax/shipping/discount integration. Check invariants: quantity is positive, discount is bounded, total is non-negative, and repeated calculation is deterministic. Use mutation testing or seeded defects to assess whether the suite detects comparison and formula errors.

Conclusion / justification

ECP and BVA cover individual ranges; decision tables cover interactions. Combining them with risk-based pairwise testing gives stronger defect detection without blindly enumerating every theoretical combination.

4. A software module has complex nested conditions and loops, but the testing team reports only partial path coverage. Evaluate the current white box testing approach and identify gaps in control flow graph construction and path testing. Analyze how inadequate identification of independent paths impacts defect detection. Propose a complete path testing strategy ensuring all critical paths are tested. Justify your decisions using coverage criteria and expected outcomes. Present your analysis in a clear and structured manner.

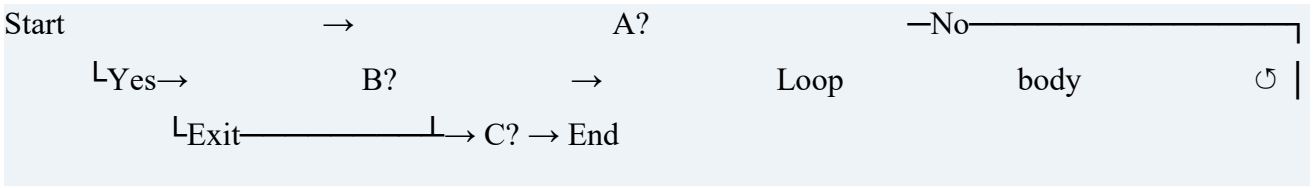
Scenario analysis

Partial path coverage often means the CFG is incomplete, nested decision outcomes were overlooked, loop behavior was not modeled, or the team equated line execution with path adequacy. These gaps can hide combinations that only occur through a particular branch sequence.

Relevant theory

A CFG models basic blocks as nodes and control transfers as directed edges. Cyclomatic complexity estimates the number of linearly independent paths: $V(G)=E-N+2P$, where E is edges, N nodes and P connected components (usually 1). Basis path testing selects independent paths that collectively exercise every edge at least once. It does not imply every possible path is tested when loops create unbounded path counts.

Conceptual flow / architecture



Redesigned test cases

Test focus	Input / scenario	Expected result
Outer condition true	Enter nested decision	Exercise inner true and false outcomes
Outer condition false	Skip nested block	Verify bypass path
Loop zero iterations	Condition false initially	No body side effects
Loop one iteration	Condition true once	Correct single-iteration result
Loop many iterations	Repeated condition true then false	Correct termination and accumulation
Nested loop boundary	Inner and outer loops at min/max counts	No off-by-one or nontermination

Improved testing strategy

Review source and requirements to identify every decision, loop exit, exception and early return. Draw the CFG and calculate V(G); design a basis set of independent paths, then add loop tests for zero, one, typical, maximum-supported and invalid iteration counts. For complex predicates, use condition coverage or MC/DC. Use timeouts and watchdogs to detect nontermination.

Coverage, architecture and important checks

Report statement, branch, basis-path and loop-boundary coverage separately. Mark infeasible paths with justification rather than claiming complete path coverage. A CFG missing exception edges or short-circuit outcomes yields a misleadingly low complexity and incomplete tests.

Conclusion / justification

Systematic CFG construction plus independent-path and loop-boundary tests makes the path strategy measurable, repeatable and more likely to expose logic and termination defects.

5. A financial application combines both black box and white box testing, yet defects continue to appear in production. Critically analyze the integration of test case design strategies used in the project. Evaluate whether test adequacy criteria and coverage metrics were properly applied. Propose an improved hybrid testing strategy combining black box techniques (ECP, BVA) and white box methods (path testing, CFG). Justify how your approach ensures better coverage and defect prevention. Conclude with a well-structured and reasoned solution.

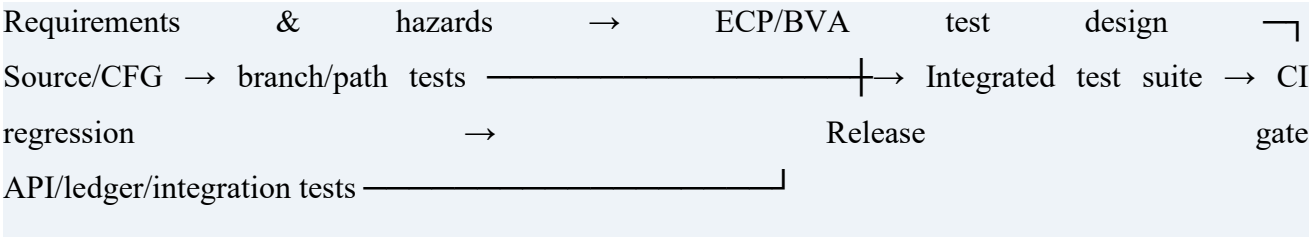
Scenario analysis

Defects in production despite using both approaches suggest the methods may have been applied separately, without traceability, risk prioritization, test oracles, integration coverage or meaningful adequacy thresholds.

Relevant theory

Black-box tests validate externally observable requirements without relying on implementation. ECP and BVA target input domains and edges. White-box CFG/path tests expose internal decision and exception paths. Neither alone is sufficient: black-box tests can miss internal dead/error paths, while white-box tests can faithfully execute code that implements the wrong requirement.

Conceptual flow / architecture



Redesigned test cases

Test focus	Input / scenario	Expected result
Transfer boundaries	min−δ, min, min+δ, max−δ, max, max+δ	Correct accept/reject result
Branch outcomes	each validation and authorization decision true/false	All outcomes exercised
Failure path	ledger unavailable after validation	Atomic rollback; clear error
Security rule	unauthorized or expired session	No transaction occurs
Concurrency	two transfers race on same balance	No overdraft or lost update
Recovery/idempotency	retry after timeout	No duplicate debit

Improved testing strategy

Create a requirements-to-risk-to-test traceability matrix. Apply ECP/BVA at API boundaries; use CFG, branch and basis-path tests for transaction logic; add integration tests for ledger, limits, authentication and audit logs. Define measurable exit criteria, such as all critical requirements tested, 100% branch coverage for critical decision code where feasible, no unresolved critical defects, and all boundary regressions passing. Add independent test oracles, code review, mutation testing and production telemetry.

Coverage, architecture and important checks

Coverage metrics are evidence, not proof of correctness. Review uncovered lines/branches, justify exclusions, and measure defect detection through mutation testing and escaped-defect analysis. Test rollback, consistency and concurrency properties in addition to nominal outputs.

Conclusion / justification

A hybrid strategy closes the gap between external correctness and internal execution. Traceability, risk-based gates and regression automation turn coverage into actionable release evidence.

6. A specialized HR payroll system calculates monthly bonuses based on years of service and performance ratings. Testers found that employees at exactly 5, 10, and 15 years were receiving incorrect payouts. Critically analyze the failure and evaluate the current test design. Apply Boundary Value Analysis (BVA) and Equivalence Class Partitioning (ECP) to redesign the test suite. Justify why these techniques are superior to exploratory testing for this specific financial logic.

Scenario analysis

Errors exactly at 5, 10 and 15 years strongly suggest threshold operators or interval definitions are wrong—for example, a band uses < where it should use ≤. Exploratory testing may notice anomalies but does not guarantee systematic tests around every service-year threshold.

Relevant theory

ECP divides service duration into bonus bands, each with a distinct rule. BVA tests immediately below, exactly at and immediately above every boundary. If service is recorded in completed years, δ may be one day or one smallest stored unit; the test design must reflect the actual data model. Performance-rating categories should also be partitioned and combined with service bands.

Conceptual flow / architecture

Read service duration + rating → Validate → Determine service band (<5 | 5–<10 | 10–<15 | ≥15) → Apply rating multiplier → Calculate bonus → Payroll/audit

Redesigned test cases

Test focus	Input / scenario	Expected result
Below 5 years	4 years, 364 days	Band below 5
Exactly 5 years	5-year anniversary	Correct 5-year band
Above 5 years	5 years + 1 day	Next applicable band
Around 10 years	10 years − 1 day; exact; +1 day	Correct band transition
Around 15 years	15 years − 1 day; exact; +1 day	Correct band transition
Rating boundaries	lowest, each threshold, highest, invalid	Correct multiplier or rejection

Combined cases	each service band × each rating class	Correct final bonus

Improved testing strategy

Confirm whether eligibility uses calendar anniversaries, completed years, or fractional service. Define inclusive/exclusive intervals explicitly. Apply BVA around 5, 10 and 15 years and ECP for each service band and rating class. Use a decision table for the cross-product of service band and rating, then test leap-day anniversaries, leave/service adjustments, rounding, retroactive changes and payroll reruns.

Coverage, architecture and important checks

Compare calculated results with independently approved examples and invariants. Verify that a rejected/failed calculation does not partially update payroll and that the audit record explains the chosen band and formula.

Conclusion / justification

ECP+BVA is superior as the core method for threshold-based financial logic because it systematically targets known risk points and is repeatable. Exploratory testing remains useful for discovering unanticipated business rules, but should supplement rather than replace it.

7. An automated temperature control system for a laboratory uses complex if-else logic to manage heaters and coolers. Post-release, the system failed to trigger the alarm during a specific sensor-malfunction sequence. Construct a Control Flow Graph (CFG) for such a logic module and evaluate the gaps in branch coverage. Propose a white-box testing strategy that ensures 100% Modified Condition/Decision Coverage (MC/DC) and justify its impact on safety-critical reliability.

Scenario analysis

An alarm failed during a sensor-malfunction sequence. Branch coverage may have executed the overall decision as true and false without proving that each Boolean condition independently affects the outcome. Safety-critical logic needs MC/DC and explicit fault-sequence tests.

Relevant theory

Modified Condition/Decision Coverage (MC/DC) requires each atomic condition to be shown to independently affect the decision outcome while other conditions are held appropriately constant. Exact tests depend on the actual expression. For illustration, let A=sensor invalid, B=temperature above safe limit, C=controller failure, and D=alarm enabled; the alarm decision might be A OR (B AND C) AND D. The real specification must define fail-safe behavior.

Conceptual flow / architecture

```
Read sensors → Sensor valid? ─No→ Enter safe state + alarm → Log
               └Yes→ Temperature high? ─No→ Normal control
                     └Yes→ Cooling available? ─Yes→ Cool
                                              └No→ Alarm + safe state
```

Redesigned test cases

Test focus	Input / scenario	Expected result
Sensor validity	valid and invalid sensor	Invalid sensor independently triggers fault handling
High temperature	below/at/above threshold	Correct heating/cooling/alarm decision
Actuator availability	cooler available/unavailable	Unavailable cooling triggers safe response
Alarm channel	enabled/failed	Failure escalates through fallback channel
Sequence	sensor fails after high-temperature reading	No stale value suppresses alarm

Improved testing strategy

Draw a CFG including normal, sensor-invalid, high-temperature, actuator-failure, alarm and recovery nodes. List each decision's atomic conditions and construct an MC/DC table proving each condition independently changes the decision result. Add boundary tests for temperature thresholds, injected sensor faults, stale data, communication loss and recovery. Verify safe-state behavior and alarm latency against requirements.

Coverage, architecture and important checks

Target 100% MC/DC for safety-related Boolean decisions where applicable, alongside statement and branch coverage. Document infeasible combinations and review them independently. Test hardware-in-the-loop timing and fault injection because source-level coverage alone cannot prove physical alarm behavior.

Conclusion / justification

MC/DC is stronger than branch coverage for complex Boolean safety logic because it demonstrates independent condition influence. Combined with fault-sequence and hardware-in-the-loop tests, it reduces the chance that a sensor fault silently bypasses the alarm.

8. A travel booking portal allows users to apply multiple promo codes, but certain combinations lead to a negative "Total Amount" due. Analyze the limitations of testing these inputs in isolation. Apply Decision Table Testing or Cause-Effect Graphing to map all valid and invalid combinations of price, promo type, and user tier. Propose an optimized test set that maximizes defect detection while minimizing redundant test cases.

Scenario analysis

Testing price, promo type and user tier in isolation misses interactions, stacking rules and order-of-application defects. Multiple promotions may over-discount the fare, apply to ineligible users or create a negative total.

Relevant theory

Decision-table testing lists conditions and resulting actions for combinations. Cause-effect graphing models logical relationships between input causes and output effects before selecting representative tests. Exact business rules must specify whether promotions stack, which discount applies first, caps, exclusions and rounding.

Conceptual flow / architecture

Base price + promo code(s) + user tier → Validate eligibility → Resolve stacking/priority → Apply capped discount → Compute total → Assert total ≥ 0

Redesigned test cases

Test focus	Input / scenario	Expected result
No promotion	positive price; no code	Base price or tier benefit per rule
Single valid promo	eligible type and tier	Expected discount
Single invalid promo	expired/ineligible code	Reject or ignore code
Two stackable promos	both eligible	Apply permitted stack/cap
Two non-stackable promos	both individually valid	Apply priority rule or reject second
Zero/negative price	0 or negative base price	Reject invalid booking price

Excess discount	discount sum exceeds price	Cap or reject; never negative total
Tier boundary	tier changes at expiry/upgrade	Use tier valid at specified booking time

Improved testing strategy

Build a compact decision table with conditions for base price valid, code valid, code stackable, tier eligible, promo expired and discount cap exceeded. Collapse columns with identical actions, then choose one test per distinct rule outcome plus high-risk combinations. Add pairwise coverage for secondary dimensions, but do not pairwise-reduce combinations that directly determine stacking or negative totals. Use property assertions such as $0 \leq \text{total} \leq \text{base price}$, unless fees or credits explicitly permit otherwise.

Coverage, architecture and important checks

Test UI, API and server-side calculation independently; never trust client-submitted discount values. Include currency precision, rounding order, duplicate codes, replay, and promotion changes between quote and checkout.

Conclusion / justification

Decision tables efficiently cover rule interactions and reduce redundant cases while preserving critical combinations. Invariants catch impossible totals even when a specific example was omitted.

9. A legacy database migration tool uses a deeply nested loop structure to validate records. The team reports high statement coverage but frequent "infinite loop" crashes in production. Evaluate the current white-box approach and identify why statement coverage was insufficient. Propose a Path Testing strategy focusing on independent paths and loop boundary conditions. Justify your approach using Cyclomatic Complexity metrics.

Scenario analysis

High statement coverage does not guarantee loop termination. A loop body may execute once and still fail when the exit condition never becomes true, an index is not updated, or a nested-loop variable is reset incorrectly.

Relevant theory

Statement coverage confirms execution, not correct control flow or termination. Basis Path Testing uses CFG and cyclomatic complexity V(G)=E−N+2P to estimate independent paths. Loop testing should include zero, one, typical, maximum and invalid iteration counts, plus tests for progress and termination invariants.

Conceptual flow / architecture

```
Start → Read batch → Records remaining? ─No→ Commit/End
                  └Yes→ Validate record → Update index/state → Next record ↺
                                  └Error→ Record failure / controlled exit
```

Redesigned test cases

Test focus	Input / scenario	Expected result
Empty dataset	0 records	Loop body skipped; clean completion
Single record	1 record	Exactly one validation
Typical batch	many records	All records processed once
Maximum batch	configured maximum	Completes within time/memory budget
Malformed record	validation exception	Controlled handling; index advances or exits
Nested-loop boundary	first/last record and inner list empty/large	No skipped/duplicate records
Non-progress guard	state/index cannot advance	Timeout/watchdog and clear failure

Improved testing strategy

Construct a complete CFG including error edges, continue/break statements, retry paths and nested loops. Calculate V(G), then choose independent paths for empty input, valid record, invalid record, retry, early exit and normal termination. Test loop counts at 0, 1, 2 and maximum-supported values; verify a progress measure decreases or index increases on every iteration. Use timeouts and memory limits in stress tests.

Coverage, architecture and important checks

Cyclomatic complexity is a path-design aid, not a guarantee that every loop behavior is covered. Track branch and basis-path coverage plus termination assertions. For nested loops, exhaustive path testing can explode; use risk-based loop boundary combinations and invariant checks.

Conclusion / justification

Statement coverage misses infinite loops because executed lines may repeat indefinitely. CFG-based independent paths, loop boundaries, progress invariants and timeout checks directly target the failure mode.

10.A mobile food delivery app features a "Real-time Tracker" that transitions through various states (Ordered, Preparing, Out for Delivery, Delivered). Users reported the app getting stuck when a driver cancels mid-delivery. Evaluate the current black-box strategy and apply State Transition Testing to redesign the scenarios. Justify how this technique addresses the logic gaps that ECP and BVA might miss in dynamic workflows.

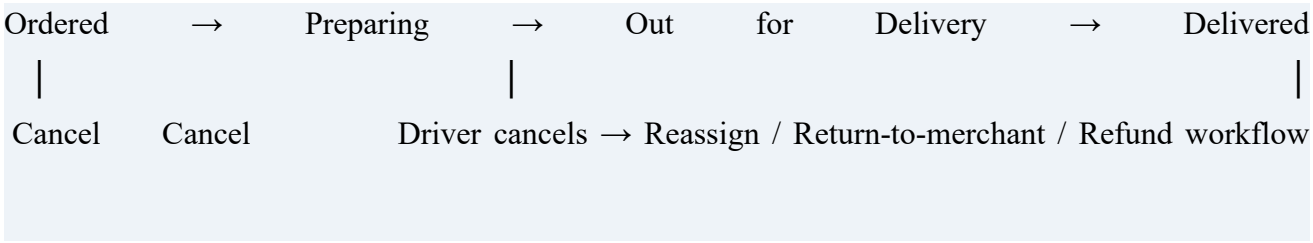
Scenario analysis

The tracker becomes stuck when a driver cancels mid-delivery, suggesting the state model lacks a cancellation transition or handles cancellation only before dispatch. ECP and BVA focus on input values, not sequences of events and states.

Relevant theory

State Transition Testing models states, events, guards and actions. It checks valid transitions, rejects invalid transitions and verifies that the system reaches a consistent terminal or recovery state. A state-transition table is particularly useful for dynamic workflows.

Conceptual flow / architecture



Redesigned test cases

Test focus	Input / scenario	Expected result
Normal flow	Ordered→Preparing→Out for Delivery→Delivered	Correct state and notifications
Cancel before preparation	Ordered + cancel	Cancel/refund per policy
Cancel during preparation	Preparing + cancel	Defined cancellation result
Driver cancellation	Out for Delivery + driver-cancel event	Reassign driver or controlled cancellation/refund; never remain stuck
Duplicate event	same cancellation delivered twice	Idempotent handling
Late event	Delivered then cancellation event	Reject/ignore and preserve terminal state
Network retry	event delayed/replayed	Eventually consistent state without duplicate action

Improved testing strategy

Define a state-transition table with current state, event, guard, next state and side effects. Cover every valid transition, each invalid transition, cancellation from each cancellable state, retries, duplicate events and out-of-order delivery. Add transition-pair coverage for two-event sequences and model-based automated tests. Verify state persistence and notification/refund side effects transactionally.

Coverage, architecture and important checks

Measure state coverage, transition coverage and transition-pair coverage; statement coverage alone is not meaningful for workflow completeness. Define terminal states and timeouts so monitoring can detect orders stuck beyond an expected duration.

Conclusion / justification

State transition testing directly exercises event sequences and missing edges, catching workflow defects that value partitioning cannot. Model-based regression makes future workflow changes safer.

11.A cybersecurity authentication module uses a multi-factor logic involving password strength, OTP timing, and IP location. Despite testing, "Brute Force" vulnerabilities were detected. Critically assess the integration of black-box and white-box methods used. Propose a Hybrid Testing strategy that combines Error Guessing with formal Path Analysis. Justify how this integrated approach enhances the security posture of the module.

Scenario analysis

Brute-force vulnerabilities can persist when tests validate successful and failed logins but omit repeated attempts, distributed sources, timing windows, lockout reset logic or combinations of password, OTP and IP signals.

Relevant theory

Black-box security tests exercise externally visible behavior and abuse cases. White-box path analysis examines authentication decisions, rate-limit checks, OTP verification, lockout counters and bypass paths. Error Guessing uses domain experience to target likely faults; it should be paired with systematic coverage and a threat model.

Conceptual flow / architecture

Request → Validate password → Rate limit/lockout? → Verify OTP + expiry → Evaluate IP/risk policy → Allow / deny / step-up → Audit event

Redesigned test cases

Test focus	Input / scenario	Expected result
Invalid password repetition	many failures from one account/IP	Rate limit or lockout policy enforced
OTP timing	valid, expired, reused, just-before/after expiry	Only valid unused OTP accepted
IP/risk variation	trusted, unfamiliar, suspicious IP	Policy and step-up applied
Parallel attempts	concurrent requests	Counters are atomic; limits cannot be raced

Recovery/reset	password reset or lockout expiry	No bypass of MFA or counters
Error-message/timing	valid vs invalid account	Avoid account enumeration where required

Improved testing strategy

Threat-model brute force, credential stuffing, OTP replay, race conditions, distributed attempts and recovery bypass. Build a CFG for each authentication decision and test independent branches and critical paths, including rate-limit-before-verification, OTP expiry and lockout recovery. Apply error guessing for counter resets, boundary timing, null/malformed tokens and concurrent requests. Use controlled security testing, secure logging, alerts and abuse-rate monitoring.

Coverage, architecture and important checks

Measure decision/branch coverage for critical authentication logic, plus security properties such as maximum attempts, OTP single-use and no authorization without all required factors. Coverage is not a substitute for penetration testing or threat modeling.

Conclusion / justification

The hybrid strategy combines systematic path evidence with attacker-minded tests. It is more effective than either ordinary functional testing or intuition-based error guessing alone.

12. An e-learning platform provides tiered access (Free, Basic, Premium) based on subscription dates and course types. Errors were found when a user upgrades exactly on the day of expiration. Evaluate the adequacy of the current testing criteria. Apply Equivalence Class Partitioning to the date ranges and subscription types. Propose a structured test design and justify its completeness in ensuring seamless user transitions.

Scenario analysis

Upgrade errors exactly on expiry day suggest ambiguity about inclusive end dates, time zones, transaction ordering or whether the old plan expires before the new plan activates.

Relevant theory

ECP groups dates into before expiry, on expiry, after expiry and invalid/future/past ranges; subscription type classes include Free, Basic and Premium. Date boundaries require precise definitions of time zone, timestamp precision and inclusive/exclusive intervals. BVA tests immediately before, exactly at and immediately after expiry.

Conceptual flow / architecture

Read account + subscription dates → Normalize timezone → Determine current entitlement → Process upgrade → Apply effective date → Recompute access → Audit

Redesigned test cases

Test focus	Input / scenario	Expected result
Before expiry	now = expiry − smallest time unit	Old tier remains active; upgrade rules applied
Exactly at expiry	now = expiry timestamp/date boundary	Expected policy consistently applied
After expiry	now = expiry + smallest unit	Expired access and upgrade handled correctly
Tier transitions	Free→Basic, Basic→Premium, Premium renewal	Correct permissions and billing
Invalid dates	end before start; malformed timezone	Reject safely

Clock/timezone	DST or different client/server zones	No unintended access gap or extension
Retry/idempotency	same upgrade request repeated	No duplicate charge or conflicting tier

Improved testing strategy

Define whether expiry is inclusive and whether dates are stored as UTC timestamps or calendar dates. Partition date ranges and subscription types, apply BVA around expiration, and use a decision table for current tier × requested tier × date status × payment status. Test entitlement checks at UI and API layers, renewal/upgrade races and billing consistency.

Coverage, architecture and important checks

Trace subscription rules to tests and verify both access permissions and charges. Include audit history and eventual-consistency checks if entitlement updates propagate asynchronously.

Conclusion / justification

Date ECP+BVA creates clear, repeatable evidence around the exact expiry boundary. Explicit timezone and interval rules eliminate ambiguity and reduce both access errors and billing defects.

13. A smart-home lighting controller uses a specific algorithm to dim lights based on ambient light sensors. The code contains several switch-case statements and complex Boolean expressions. Evaluate the risks of relying solely on statement coverage for this module. Apply Branch and Condition Coverage techniques to identify missing logical paths. Design an improved white-box strategy and justify how it reduces the risk of hardware-software integration failures.

Scenario analysis

Statement coverage can execute each switch case or line while failing to test every switch outcome or Boolean condition. This may leave sensor ranges, manual overrides, hardware fault flags or combined conditions untested.

Relevant theory

Branch coverage exercises each decision outcome; condition coverage requires each atomic Boolean condition to take true and false values. Neither necessarily proves that each condition independently changes the overall decision, which is why MC/DC may be useful for complex safety-relevant expressions. Switch statements should include every case and default behavior.

Conceptual flow / architecture

Read ambient sensor → Sensor valid? ─No→ Safe brightness + fault log
└Yes→ Manual override? ─Yes→ Apply manual level
└No→ Switch ambient band → Select dimming level → Command light → Verify feedback

Redesigned test cases

Test focus	Input / scenario	Expected result
Switch cases	dark, dim, normal, bright, unknown	Each case and default exercised
Sensor validity	valid/invalid	Safe fallback on invalid reading
Manual override	on/off combined with ambient bands	Override precedence is correct
Boolean conditions	each condition true and false	Condition outcomes covered

Hardware feedback	command succeeds/fails	Retry or fault indication as specified

Improved testing strategy

Build a CFG including each switch case, default, sensor fault, override and hardware feedback path. Create a condition matrix for complex expressions and test branch plus condition coverage; apply MC/DC to critical expressions where feasible. Test sensor boundaries, noisy readings, stale data, actuator disconnects and rapid changes. Use hardware-in-the-loop tests to validate the command reaches the physical device.

Coverage, architecture and important checks

Statement coverage can miss unvisited cases and condition combinations. Report switch-case, branch, condition and integration coverage separately; do not treat one metric as proof of system correctness.

Conclusion / justification

Branch and condition testing exposes decision gaps, while hardware-in-the-loop testing catches interface failures that code coverage cannot. Together they improve reliable behavior under real sensor and actuator conditions.

14. A logistics management system calculates shipping costs based on weight, destination zone, and delivery speed. Testers failed to detect errors when weights were entered as zero or negative values. Analyze the failure of the current input validation testing. Apply Boundary Value Analysis (Robustness Testing) to include invalid boundaries. Justify the necessity of testing "negative scenarios" for robust system performance.

Scenario analysis

Failure to test zero and negative weights leaves invalid inputs able to enter shipping calculations. This may produce negative charges, divide-by-zero behavior, incorrect weight bands or inconsistent carrier requests.

Relevant theory

Robust Boundary Value Analysis tests valid boundaries and values just outside them. If valid weight is $w \geq w_min$ and $w \leq w_max$, test $w_min-\delta$, w_min, $w_min+\delta$, $w_max-\delta$, w_max and $w_max+\delta$. Include zero and negative values explicitly, with δ based on the input precision.

Conceptual flow / architecture

Receive weight/zone/speed → Validate weight > 0 and within limit? ─No→ Reject with clear error
└Yes→ Determine zone + speed tariff → Calculate cost → Round → Return quote

Redesigned test cases

Test focus	Input / scenario	Expected result
Negative weight	$-\delta$	Reject; no carrier request
Zero weight	0	Reject unless specification defines a special shipment
Minimum valid	w_min	Correct minimum tariff
Just above minimum	w_min+δ	Correct band/charge
Maximum valid	w_max	Accept if inclusive
Above maximum	w_max+δ	Reject or route to special handling
Zone/speed combinations	each zone × delivery speed	Correct tariff and surcharge

Precision/rounding	fractional weights near band edges	Documented rounding behavior

Improved testing strategy

Use robust BVA on weight and ECP for destination zone, delivery speed and invalid formats. Create a decision table for weight band × zone × speed. Test API validation so client-side checks cannot be bypassed. Assert that invalid weight never produces a quote or external shipment booking and that valid costs are non-negative and reproducible.

Coverage, architecture and important checks

Include null, text, overflow, unit mismatch (kg versus lb), and extremely large values. Verify logging and error responses do not expose internal details. Regression-test tariff changes and rounding.

Conclusion / justification

Negative scenarios are essential because real systems receive malformed, hostile and integration-generated inputs. Robust BVA catches boundary validation defects early and protects cost integrity.

15. A medical imaging software processes large datasets using parallel processing loops. Inconsistent results occur only when the data size is a prime number. Evaluate the testing team's reliance on random testing for such a specialized mathematical module. Propose a structured White-Box Testing approach that incorporates Basis Path Testing. Justify your solution by explaining how it ensures mathematical accuracy and system stability.

Scenario analysis

Random tests may rarely generate prime-sized datasets or the precise partition/tile layout that triggers a parallel-loop defect. Inconsistent results may arise from off-by-one bounds, uneven chunking, race conditions or reduction-order differences.

Relevant theory

Basis Path Testing derives independent paths from the CFG using cyclomatic complexity. White-box tests should be complemented by mathematical properties, deterministic reference outputs, concurrency stress and metamorphic tests. Prime sizes are a useful targeted class because they often produce uneven partitions, but the team must not assume primality alone is the root cause.

Conceptual flow / architecture

Load dataset → Validate dimensions → Compute partitions/chunks → Parallel workers process chunks → Merge/reduce results → Compare invariants/reference → Save output

Redesigned test cases

Test focus	Input / scenario	Expected result
Small prime size	3, 5, 7 elements/pixels as meaningful	Correct handling of uneven partition
Composite size	4, 6, 8 elements	Reference output matches
Boundary sizes	0, 1, worker-count−1, worker-count, worker-count+1	No skipped or duplicated elements
Large prime size	prime dimensions near supported maximum	Stable completion and correct result
Worker failure	one chunk fails or retries	No partial/corrupt output

Concurrency repeat	same input across runs/thread counts	Deterministic result within tolerance
Invalid dimensions	negative/overflow/inconsistent shape	Safe rejection

Improved testing strategy

Construct a CFG for validation, partitioning, worker dispatch, error/retry, merge and output paths. Calculate V(G) and test a basis set of independent paths, then specifically vary prime/composite sizes, dimensions relative to worker count, and loop bounds. Use a trusted serial implementation as an oracle; compare pixel-wise or with medically justified tolerances. Add race detection, thread-count variation, randomized tests with recorded seeds and metamorphic properties such as partitioning not changing the final result.

Coverage, architecture and important checks

Track branch and basis-path coverage, loop-boundary coverage, data-size classes and concurrency configurations. For parallel code, source coverage alone cannot prove race freedom or numerical accuracy; use sanitizers, deterministic scheduling where possible, stress testing and domain validation.

Conclusion / justification

Random testing should remain supplementary, but structured basis-path and loop-boundary tests target the suspected defect mechanism. Prime-sized cases plus independent reference comparisons improve reproducibility, mathematical confidence and stability.

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Problem Understanding & Context Analysis	25	Thoroughly understands the scenario with accurate interpretation of all requirements	Clearly understands the scenario with minor gaps	Basic understanding; some aspects of the scenario unclear	Poor understanding or misinterpretation of the scenario
Application of Concepts	30	Applies appropriate concepts effectively to solve complex real-world problems	Applies relevant concepts correctly with minor errors	Applies basic concepts but with limited accuracy	Fails to apply appropriate concepts
Analytical & Decision-Making Skills	20	Demonstrates strong analysis and selects optimal solutions with justification	Good analysis with reasonable solutions	Limited analysis; solutions lack justification	Poor or no analysis; incorrect decisions
Solution Design & Approach	15	Well-structured, innovative, and feasible solution approach	Logical and structured solution with minor gaps	Basic solution with limited structure or feasibility	Unclear or incorrect solution approach
Clarity, Justification & Presentation	10	Clear, well-justified explanation with strong reasoning	Clear explanation with some justification	Limited clarity and weak justification	Poorly explained with no justification